

```

----- archivo todoCrud.js
-----
import {
  addalumno,
  getalumnosCollection,
  deletealumnoCollection,
  getalumnoCollection,
  updatealumnoCollection,
  marcarParaBorrar,
  getCursosCollection
} from "../firebase.js";

/**
 * Función para refrescar la tabla después de cualquier acción
 */
async function refrescarTabla() {
  if (document.getElementById("tablaalumnosSinHuella")) {
    await mostraralumnosSinHuella();
  }
  if (document.getElementById("tablaalumnos")) {
    await mostraralumnosEnHTML();
  }
}

window.miModal = async function (idModal, idalumno = "") {
  try {
    // 1. Limpieza de seguridad
    const existing = document.getElementById(idModal);
    if (existing) {
      const ins = bootstrap.Modal.getInstance(existing);
      if (ins) ins.dispose();
      existing.remove();
    }
    document.querySelectorAll('.modal-backdrop').forEach(el => el.remove());

    // 2. Cargar el PHP
    let url = "";
    switch (idModal) {
      case "agregaralumnoModal": url = "modales/modalAdd.php"; break;
      case "detallealumnoModal": url = "modales/modalDetalles.php"; break;
      case "editaralumnoModal": url = "modales/modalEditar.php"; break;
      case "eliminaralumnoModal": url = "modales/modalDelete.php"; break;

      default: return;
    }

    const response = await fetch(url);
    const text = await response.text();

    // 3. Convertir texto a elementos y FILTRAR solo el DIV del modal

```

```

const parser = new DOMParser();
const htmlDoc = parser.parseFromString(text, 'text/html');

// Esto busca el div.modal real ignorando comentarios y textos
const modalElement = htmlDoc.querySelector('.modal');

if (!modalElement) {
  console.error("No se encontró un div con clase 'modal' en el archivo
cargado.");
  return;
}

// 4. Inyectar y mostrar
document.body.appendChild(modalElement);

const myModal = new bootstrap.Modal(modalElement);
myModal.show();

    if (idModal === "agregaralumnoModal") {
      await cargarCursosEnModal(); // <--- Nueva llamada
    } else if (idModal === "detallealumnoModal") {
      await cargarDetallealumno(idalumno);
    }

    // 5. Cargar datos según corresponda
    if (idModal === "detallealumnoModal") {
      await cargarDetallealumno(idalumno);
    } else if (idModal === "editaralumnoModal") {
      await getalumnoUpdateCollection(idalumno);
    } else if (idModal === "eliminaralumnoModal") {
      const btn = modalElement.querySelector("#confirmDeleteBtn");
      if (btn) btn.onclick = async () => {
        await eliminaralumno(idalumno);
        //await solicitarBorradoHuella(idalumno);
        myModal.hide();
      };
    }

  } catch (error) {
    console.error("Error al abrir modal:", error);
  }
};

// async function mostraralumnosSinHuella() {
//   try {
//     const queryCollection = await getalumnosCollection();
//     const tablaalumnos = document.querySelector("#tablaalumnos tbody");
//     if (!tablaalumnos) return;

```

```

//      tablaalumnos.innerHTML = "";

//      queryCollection.forEach((doc) => {

//          const alumno = doc.val();

//          const id = doc.key;
//          if (alumno.borrar === "Si") { return; }
//          const fila = document.createElement("tr");
//          fila.id = id;
//          fila.innerHTML = `
//              <td>${alumno.curso}</td>
//              <td>${alumno.nombre}</td>
//              <td>${alumno.dni}</td>
//              <td>${alumno.obs}</td>
//              <td>
//                  <a title="Ver detalles" href="#"
onclick="window.miModal('detallealumnoModal','${id}')" class="btn btn-success
btn-sm">
//                      <i class="bi bi-binoculars"></i>
//                  </a>
//                  <a title="Editar" href="#"
onclick="window.miModal('editaralumnoModal','${id}')" class="btn btn-warning
btn-sm">
//                      <i class="bi bi-pencil-square"></i>
//                  </a>
//                  <a title="Eliminar" href="#"
onclick="window.miModal('eliminaralumnoModal','${id}')" class="btn btn-danger
btn-sm">
//                      <i class="bi bi-trash"></i>
//                  </a>
//              </td>
//          `;
//          tablaalumnos.appendChild(fila);
//      });
//  } catch (error) {
//      console.error("Error al obtener los alumnos:", error);
//  }
// }

/**
 * Función para mostrar alertas (Usando iziToast)
 * Movida arriba para que esté disponible cuando se necesite.
 */
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (typeof iziToast === 'undefined') {
        console.warn("iziToast no está cargado, usando alert normal:", mensaje);
        alert(mensaje);
        return;
    }
}

```

```

    if (tipoToast === "success") {
        iziToast.success({
            timeout: 5000,
            icon: "bi bi-check-circle-fill",
            title: "¡Éxito!",
            message: mensaje,
            position: "topRight"
        });
    } else if (tipoToast === "error" || tipoToast === "warning") {
        iziToast.error({
            timeout: 5000,
            icon: "bi bi-x-circle-fill",
            title: "¡Error!",
            message: mensaje,
            position: "topRight"
        });
    }
};

// ... resto de tus funciones (addNuevoalumno, etc.)

/**
 * CREATE: Agrega un nuevo alumno
 */
window.addNuevoalumno = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioalumno");
    const formData = new FormData(formulario);
    const dataJSON = Object.fromEntries(formData.entries());

    try {
        await addalumno(dataJSON.curso, dataJSON.nombre, dataJSON.dni,
dataJSON.obs, -1); //-1 sin id de huella asignando
        formulario.reset();

        const modalElt = document.getElementById("agregaralumnoModal");
        bootstrap.Modal.getInstance(modalElt).hide();

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno registrado
con éxito!" });
        await refrescarTabla();
    } catch (error) {
        console.error("Error al añadir:", error);
    }
};

/**
 * READ ONE: Detalles del alumno

```

```

*/
async function cargarDetallealumno(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      const contenedor = document.querySelector("#detallealumnoContenido
ul");
      if (!contenedor) return;

      contenedor.innerHTML = `
        <li class="list-group-item"><b>Curso:</b> ${data.curso ||
"N/A"}</li>
        <li class="list-group-item"><b>Apellido y Nombre:</b> ${data.nombre
|| "N/A"}</li>
        <li class="list-group-item"><b>DNI:</b> ${data.dni || "N/A"}</li>
        <li class="list-group-item"><b>Obs:</b> ${data.obs || "N/A"}</li>
      `;
    }
  } catch (error) {
    console.error("Error al cargar detalles:", error);
  }
}

/**
 * UPDATE: Cargar datos en el formulario de edición
 */
async function getalumnoUpdateCollection(id) {
  try {
    const alumnoDoc = await getalumnoCollection(id);
    if (alumnoDoc.exists()) {
      const data = alumnoDoc.val();
      document.querySelector('[name="idalumno"]').value = id;
      document.querySelector('[name="curso"]').value = data.curso;
      document.querySelector('[name="nombre"]').value = data.nombre;
      document.querySelector('[name="dni"]').value = data.dni;
      document.querySelector('[name="obs"]').value = data.obs;
    }
  } catch (error) {
    console.error("Error al obtener datos para editar:", error);
  }
}

/**
 * UPDATE: Guardar cambios del alumno
 */
window.actualizaralumno = async function (event) {
  event.preventDefault();
  const formulario = document.querySelector("#formularioalumnoEdit");
  const formData = new FormData(formulario);

```

```

const { idalumno, ...datosNuevos } = Object.fromEntries(formData.entries());

try {
  await updatealumnoCollection(idalumno, datosNuevos);

  const modalElt = document.getElementById("editaralumnoModal");
  bootstrap.Modal.getInstance(modalElt).hide();

  window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Alumno
actualizado!" });
  await refrescarTabla();
} catch (error) {
  console.error("Error al actualizar:", error);
}
};

/**
 * DELETE: Lógica principal de borrado
 */

async function eliminaralumno(id) {
  try {
    const snapshot = await getalumnosCollection(); // O la función que uses
    para traer el alumno
    // Nota: Si puedes, usa getalumnoById(id) para ser más eficiente

    // Buscamos los datos del alumno específico en el snapshot o volvemos a
    consultar
    const alumnoSnapshot = await getalumnoCollection(id);
    if (!alumnoSnapshot.exists()) return;
    const alumno = alumnoSnapshot.val();

    if (alumno.huellaId === "-1" || alumno.huellaId === -1) {
      // CASO A: Sin huella -> Borrado físico directo
      await deletealumnoCollection(id);
      window.mostrarAlerta({ tipoToast: "success", mensaje: "Alumno eliminado
correctamente" });
    } else {
      // CASO B: Con huella -> Usamos tu función UPDATE de firebase.js
      // Pasamos el ID y el objeto con el campo que queremos cambiar
      await updatealumnoCollection(id, { borrar: "Si" });

      window.mostrarAlerta({
        tipoToast: "success",
        mensaje: "Enviando orden de borrado al sensor ESP"
      });
    }
  }
}

```

```

        await refrescarTabla();
    } catch (error) {
        console.error("Error en eliminaralumno:", error);
        window.mostrarAlerta({ tipoToast: "error", mensaje: "Error al procesar la solicitud" });
    }
}

```

// Inicialización

```

window.addEventListener("DOMContentLoaded", mostraralumnosEnHTML);

```

```

async function cargarCursosEnSelect() {
    const select = document.getElementById("filtroCurso");
    const cursosSnap = await getCursosCollection();

    cursosSnap.forEach(doc => {
        const cursoData = doc.val();
        const option = document.createElement("option");
        option.value = cursoData.Curso; // El campo de tu DB
        option.textContent = cursoData.Curso;
        select.appendChild(option);
    });
}

```

```

async function mostraralumnosEnHTML() {
    try {
        const queryCollection = await getalumnosCollection();
        const tablaalumnos = document.querySelector("#tablaalumnos tbody");
        const filtroCurso = document.getElementById("filtroCurso").value;
        const busqueda =
document.getElementById("buscarAlumno").value.toLowerCase();
        const spanContador = document.getElementById("contadorAlumnos"); // El
nuevo span

```

```

        if (!tablaalumnos) return;
        tablaalumnos.innerHTML = "";

```

```

        let contador = 0; // Inicializamos el contador

```

```

        queryCollection.forEach((doc) => {
            const alumno = doc.val();
            const id = doc.key;

            // 1. Filtro de borrado lógico
            if (alumno.borrar === "Si") return;

            // 2. Filtro por Curso
            if (filtroCurso && alumno.curso !== filtroCurso) return;

            // 3. Filtro por Apellido o DNI

```

```

const coincideNombre = alumno.nombre.toLowerCase().includes(busqueda);
const coincideDNI = alumno.dni.toString().includes(busqueda);
if (busqueda && !coincideNombre && !coincideDNI) return;

// Si llegó aquí, el alumno pasó los filtros
contador++;

// Construcción de la fila
const fila = document.createElement("tr");
fila.id = id;
fila.innerHTML = `
    <td>${alumno.curso}</td>
    <td>${alumno.nombre}</td>
    <td>${alumno.dni}</td>
    <td>${alumno.obs}</td>
    <td>
        <a onclick="window.miModal('detallealumnoModal','${id}')"
class="btn btn-success btn-sm"><i class="bi bi-binoculars"></i></a>
        <a onclick="window.miModal('editaralumnoModal','${id}')"
class="btn btn-warning btn-sm"><i class="bi bi-pencil-square"></i></a>
        <a onclick="window.miModal('eliminaralumnoModal','${id}')"
class="btn btn-danger btn-sm"><i class="bi bi-trash"></i></a>
    </td>`;
    tablaalumnos.appendChild(fila);
});

// Actualizamos el número en el HTML al terminar el bucle
if (spanContador) {
    spanContador.textContent = contador;
}

} catch (error) {
    console.error("Error al obtener alumnos:", error);
}
}

window.addEventListener("DOMContentLoaded", async () => {
    // Si existe la tabla de "Sin Huella", cargamos esa función
    if (document.getElementById("tablaalumnosSinHuella")) {
        await mostraralumnosSinHuella();
    }

    // Si existe la tabla normal (listado.html), cargamos los filtros normales
    if (document.getElementById("tablaalumnos")) {
        await cargarCursosEnSelect();
        await mostraralumnosEnHTML();

        document.getElementById("buscarAlumno").addEventListener("input",
mostraralumnosEnHTML);
    }
});

```



```

        </td>
    `;
    tablaalumnos.appendChild(fila);
  });
} catch (error) {
  console.error("Error al obtener los alumnos sin huella:", error);
}
}
async function cargarCursosEnModal() {
  const select = document.getElementById("selectCursoModal");
  if (!select) return;

  try {
    const cursosSnap = await getCursosCollection();
    // Limpiar el "Cargando..." y dejar la opción por defecto
    select.innerHTML = '<option selected value="">Seleccione</option>';

    cursosSnap.forEach(doc => {
      const cursoData = doc.val();
      const option = document.createElement("option");
      // Ajusta 'Curso' al nombre exacto del campo en tu RTDB
      option.value = cursoData.curso;
      option.textContent = cursoData.curso;
      select.appendChild(option);
    });
  } catch (error) {
    console.error("Error al cargar cursos en modal:", error);
    select.innerHTML = '<option value="">Error al cargar</option>';
  }
}

```

----- archivo CrudCurso -----

```

import {
  addCurso,
  getCursosCollection,
  deleteCursoCollection,
  getCursoCollection,
  updateCursoCollection,
  getPreceptoresCollection,
} from "../../assets/js/firebase.js";

/**
 * Función para levantar Venta Modal
 */
window.miModal = async function (idModal, idCurso = "") {
  try {
    await validarModal(idModal);

    let url = "";
    switch (idModal) {

```

```

    case "agregarCursoModal":
        url = "modales/modalAddCurso.php";
        break;
    case "detalleCursoModal":
        url = "modales/modalDetallesCurso.php";
        break;
    case "editarCursoModal":
        url = "modales/modalEditarCurso.php";
        break;
    case "eliminarCursoModal":
        url = "modales/modalDeleteCurso.php";
        break;
    default:
        throw new Error(`El idModal '${idModal}' no es válido`);
}

const response = await fetch(url);

if (!response.ok) {
    throw new Error("Error al cargar la modal");
}

// response.text() es un método en programación que se utiliza para obtener el
contenido de texto de una respuesta HTTP
const data = await response.text();

// Crear un elemento div para almacenar el contenido de la modal
const modalContainer = document.createElement("div");
modalContainer.innerHTML = data;

// Agregar la modal al documento actual
document.body.appendChild(modalContainer);

// Mostrar la modal
const modalElement = modalContainer.querySelector(`#${idModal}`);
const myModal = new bootstrap.Modal(modalElement);
myModal.show();
if (idModal === "agregarCursoModal") { // Ajusta al ID real de tu modal de curso
    await cargarPreceptoresEnModal();
} else if (idModal === "agregaralumnoModal") {
    await cargarCursosEnModal();
}

if (idModal === "detalleCursoModal") {
    await cargarDetalleCurso(idCurso);
} else if (idModal === "editarCursoModal") {
    await getCursoUpdateCollection(idCurso);
} else if (idModal === "eliminarCursoModal") {
    let DeleteBtn = document.querySelector("#confirmDeleteBtn");
    DeleteBtn.addEventListener("click", async () => {
        await eliminarCurso(idCurso);
    });
}

```

```

    });
  }
} catch (error) {
  console.error(error);
}
};

//Función para validar si existe una modal abierta
async function validarModal(idModal) {
  const existingModal = document.getElementById(idModal);
  if (existingModal) {
    const modal = bootstrap.Modal.getInstance(existingModal);
    if (modal) {
      modal.hide();
    }
    existingModal.remove();
  }
}

/**
 * Función para obtener todas las colecciones
 */
async function mostrarCursosEnHTML() {
  try {
    const snapshot = await getCursosCollection(); // Llamada a RTDB
    const tablaCursos = document.querySelector("#tablaCursos tbody");
    tablaCursos.innerHTML = "";

    if (snapshot.exists()) {
      // RTDB usa forEach sobre el snapshot directamente
      snapshot.forEach((childSnapshot) => {
        const id = childSnapshot.key; // El ID generado por push()
        const Curso = childSnapshot.val(); // AQUÍ: usamos .val() en lugar de
        .data()

        const fila = document.createElement("tr");
        fila.id = id;
        fila.innerHTML = `
          <td>${Curso.curso || "N/A"}</td>
          <td>${Curso.ubicacion || "N/A"}</td>
          <td>${Curso.capacidad || "N/A"}</td>
          <td>${Curso.obs || "N/A"}</td>
          <td>
            <button onclick="window.miModal('detalleCursoModal','${id}')"
class="btn btn-success"><i class="bi bi-binoculars"></i></button>
            <button onclick="window.miModal('editarCursoModal','${id}')" class="btn
btn-warning"><i class="bi bi-pencil-square"></i></button>
            <button onclick="window.miModal('eliminarCursoModal','${id}')"
class="btn btn-danger"><i class="bi bi-trash"></i></button>
          </td>

```

```

        `;
        tablaCursos.appendChild(fila);
    });
}
} catch (error) {
    console.error("Error al obtener los Cursos:", error);
}
}

window.addEventListener("DOMContentLoaded", mostrarCursosEnHTML);

/**
 * Función para agregar un nuevo Curso
 */

window.addNuevoCurso = async function (event) {
    event.preventDefault();

    const formulario = document.querySelector("#formularioCursoEdit");
    if (!formulario) return;

    const formData = new FormData(formulario);

    // Obtenemos los valores usando el atributo 'name' del HTML
    const curso = formData.get("curso");
    const ubicacion = formData.get("ubicacion");
    const capacidad = formData.get("capacidad");
    const obsCurso = formData.get("obsCurso");
    const preceptorID = formData.get("preceptorID"); // Coincide con el name del
select

    console.log("Datos capturados:", { curso, preceptorID });

    if (!preceptorID || preceptorID === "") {
        alert("Por favor, seleccione un preceptor");
        return;
    }

    try {
        // Usamos preceptorID (con D mayúscula como en tu firebase.js)
        await addCurso(curso, ubicacion, capacidad, obsCurso, preceptorID);

        formulario.reset();

        // Cerrar modal correctamente
        const modalElt = document.getElementById('agregarCursoModal');
        const modal = bootstrap.Modal.getInstance(modalElt);
        if (modal) modal.hide();

        await mostrarCursosEnHTML();
    }
};

```

```

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso creado con éxito!" });
    } catch (error) {
        console.error("Error al crear el curso:", error);
    }
};

```

```

/**
 * Función para cargar y mostrar los detalles del Curso en la modal
 */
async function cargarDetalleCurso(id) {
    try {
        const snapshot = await getCursoCollection(id); // Calling the fixed function above

        if (snapshot.exists()) {
            const CursoData = snapshot.val(); // USE .val() instead of .data()
            const { curso, ubicacion, capacidad, obs } = CursoData;

            const ulDetalleCurso = document.querySelector("#detalleCursoContenido ul");
            ulDetalleCurso.innerHTML = `
                <li class="list-group-item"><b>Curso:      </b> ${curso      || "No disponible"}</li>
                <li class="list-group-item"><b>Ubicación:</b> ${ubicacion || "No disponible"}</li>
                <li class="list-group-item"><b>Capacidad:</b> ${capacidad || "No disponible"}</li>
                <li class="list-group-item"><b>Obs:         </b> ${obs         || "No disponible"}</li>
            `;
        } else {
            console.log("No se encontró ningún Curso con el ID:", id);
        }
    } catch (error) {
        console.error("Error al mostrar detalles del Curso", error);
    }
}

```

```

/**
 * Buscar Curso a editar
 */
async function getCursoUpdateCollection(id) {
    try {
        const CursoDoc = await getCursoCollection(id);
        if (CursoDoc.exists()) {
            const CursoData = CursoDoc.val();

```

```

        const { curso, ubicacion, capacidad, obs } = CursoData;
        document.querySelector("#idCurso").value = id;
        document.querySelector("#curso").value = curso;
        document.querySelector("#ubicacion").value = ubicacion;
        document.querySelector("#capacidad").value = capacidad;
        document.querySelector("#obsCurso").value = obs;
    } else {
        console.log("No se encontró ningún Curso con el ID:", id);
    }
} catch (error) {
    console.error("Error al obtener los detalles del Curso:", error);
}
}

/**
 * Función para actualizar el Curso
 */
window.actualizarCurso = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioCursoEdit");
    const formData = new FormData(formulario);

    // Convertir FormData a un objeto JSON
    const formDataJSON = {};
    formData.forEach((value, key) => {
        //console.log(key, value);
        formDataJSON[key] = value;
    });

    const { idCurso, curso, ubicacion, capacidad, obsCurso } = formDataJSON;
    try {
        await updateCursoCollection(idCurso, { curso, ubicacion, capacidad,
obs:obsCurso});
        formulario.reset();

        setTimeout(() => {
            $("#editarCursoModal").css("opacity", "");
            $("#editarCursoModal").modal("hide");
        }, 300);

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso actualizado
correctamente!" });
        await mostrarCursosEnHTML();
    } catch (error) {
        console.log(error);
    }
};

```

```

/**
 * Función para borrar un Curso, una colleccion
 */
async function eliminarCurso(id) {
  try {
    await deleteCursoCollection(id);
    document.querySelector(`#${id}`).remove();
    mostrarAlerta({ tipoToast: "success", mensaje: "Curso eliminado correctamente"
});
  } catch (error) {
    console.error("Error al borrar el Curso:", error);
    mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar el Curso" });
  }
}

/**
 * Función para mostrar alertas
 */
iziToast.settings({
  timeout: 10000,
  resetOnHover: true,
  // icon: '', // icon class
  transitionIn: "flipInX",
  transitionOut: "flipOutX",
  position: "topRight", // bottomRight, bottomLeft, topRight, topLeft, topCenter,
bottomCenter, center
  onOpening: function () {
    console.log("Alerta abierta!");
  },
  onClosing: function () {
    console.log("Alerta cerrada!");
  },
});
window.mostrarAlerta = function ({ tipoToast, mensaje }) {
  if (tipoToast == "success") {
    iziToast.success({
      timeout: 5000,
      icon: tipoToast == "success" ? "bi bi-check-circle-fill" : "bi
bi-x-circle-fill",
      title: tipoToast == "success" ? "¡Éxito!" : "¡Error!",
      message: mensaje,
    });
  } else if (tipoToast == "warning") {
    iziToast.success({
      timeout: 5000,

```



```

        icon: tipo == "success" ? "bi bi-check-circle-fill" : "bi bi-x-circle-fill",
        title: tipo == "success" ? "¡Éxito!" : "¡Error!",
        message: mensaje,
    });
}
};

async function cargarPreceptoresEnModal() {
    const select = document.getElementById("selectPreceptorModal");
    if (!select) return;

    try {
        const preceptorSnap = await getPreceptoresCollection();
        select.innerHTML = '<option selected value="">Seleccione
Preceptor</option>';

        if (preceptorSnap.exists()) {
            preceptorSnap.forEach(doc => {
                const data = doc.val();
                const option = document.createElement("option");

                // IMPORTANTE: El value ahora es el ID (la clave de Firebase)
                option.value = doc.key;
                // Lo que ve el usuario sigue siendo el nombre
                option.textContent = data.nombre;

                select.appendChild(option);
            });
        }
    } catch (error) {
        console.error("Error:", error);
    }
}

```